



Universidad
Indoamérica

UNIDAD 7 · FUNDAMENTOS DE SISTEMAS OPERATIVOS

Hilos y concurrencia

Universidad Tecnológica Indoamérica

Laboratorio TD6

Un hilo: una ejecución secuencial dentro de un proceso

- También "proceso ligero": varias ejecuciones en un mismo proceso
- Todos comparten el **mismo espacio de direcciones**
- **No hay protección de memoria** entre hilos
- Un proceso = memoria virtual (protección) + hilos (ejecución paralela)

Qué se comparte, con direcciones reales

proceso · 8 hilo(s)

código

datos: shared

0x650ff476602c = 8

heap

descriptores 0 1 2

compartido por todos los hilos

hilo 1

pila

local 0x31fee74

registros: PC, SP...

hilo 2

pila

local 0x19fbe74

registros: PC, SP...

hilo 3

pila

local 0xbffee74

registros: PC, SP...

hilo 4

pila

local 0x29fde74

registros: PC, SP...

hilo 5

pila

local 0x21fce74

registros: PC, SP...

hilo 6

pila

local 0x11fae74

registros: PC, SP...

hilo 7

pila

local 0x09f9e74

registros: PC, SP...

hilo 8

pila

local 0xb7fde74

registros: PC, SP...

pilas separadas por 8,004 MiB (8 MiB + 4 KiB de guarda)

Programas con varios hilos

- Sistemas embebidos: un programa, operaciones concurrentes
- El propio núcleo: miles de hilos del núcleo
- Bases de datos y servidores de red (unidad 8)
- Programas paralelos: OpenMP, un hilo por núcleo

Hoy además

Navegadores (hilos por pestaña y por tarea), motores de juegos, la inferencia de modelos de lenguaje en CPU (unidad 9) y Python, que desde la versión 3.13 ofrece una variante sin el cerrojo global (GIL).

Crear un hilo es más barato... ¿cuánto?

Salida real

```
$ ./create_cost 2000  
2000 threads:      41.2 us per pthread_create + pthread_join  
2000 processes:   248.8 us per fork + waitpid  
a process costs 6.0 times a thread
```

- Sin páginas que copiar: solo una pila nueva y una estructura de tarea
- El sistema planifica cada hilo: **no supongas ningún orden de ejecución**

Crear, terminar y esperar un hilo

C

```
int  pthread_create(pthread_t *t, const pthread_attr_t *attr, /* NULL: defaults */
                  void *(*start)(void *), void *arg);
void pthread_exit(void *retval); /* = return retval from start; never returns */
int  pthread_join(pthread_t t, void **retval); /* PTHREAD_CANCELED if t was cancelled */
int  pthread_detach(pthread_t t); /* nobody will join t: freed when it ends */
```

- `attr` elige, por ejemplo, *joinable* (por omisión) o *detached*: se prepara con `pthread_attr_init` y `pthread_attr_setdetachstate`
- Un hilo que termina **no libera** su pila ni su estado hasta que alguien hace `join` (o si es *detached*): si no, fuga de memoria, como un zombi
- `join` suspende a quien llama hasta que `t` termina y recoge su `retval`; no se puede hacer `join` de un hilo *detached*
- En Linux, `pthread_create` usa la llamada `clone`: mismo espacio de direcciones, mismos descriptores

pthread_create, pthread_exit, pthread_join

C tds/td6/threads.c

```
for (int i = 0; i < NTHREADS; i++) {  
    int rc = pthread_create(&tids[i], NULL, start, (void *)(&i));  
    exit_if(rc != 0, "pthread_create"); /* pthread functions return the error code */  
}  
for (int i = 0; i < NTHREADS; i++)  
    exit_if(pthread_join(tids[i], NULL) != 0, "pthread_join");
```

Salida real

```
$ ./threads  
thread 57632 of process 357789: shared[0x650ff476602c] = 1, local[0x7626e31fee74] = 1  
thread 79520 of process 357789: shared[0x650ff476602c] = 4, local[0x7626e19fbe74] = 1  
...
```

Correcciones (diap. 137–140): `-pthread`, no `-lthread`; `pthread_*` devuelve 0 o el número de error, no `-1` con `errno`.

Hilos y señales

Las señales son **de todo el proceso**. Para hilos:

PROCESO	HILOS
<code>kill()</code>	<code>pthread_kill()</code>
<code>sigprocmask()</code>	<code>pthread_sigmask()</code>
<code>sigsuspend()</code>	<code>sigwait()</code> (un hilo dedicado a esperar señales)

Concurrencia y memoria compartida: dolores de cabeza

Casi siempre los hilos tocan datos distintos y el orden no importa. Pero con `x = 1` inicialmente:

```
c
/* thread 1 */      /* thread 2 */
x = x + 2;           x = x * 3;
```

¿Qué valores puede tener `x`? → **no determinista**.

- Vale igual para hilos y para procesos con memoria compartida (unidad 6)
- Errores difíciles de reproducir: aparecen una vez cada millones
- Un ejemplo peor (diapositiva 146):
`while (x < 1000) x++;` contra
`while (x > -1000) x--;` puede ganar cualquiera... o no terminar nunca

Los 20 intercalados, uno por uno

hilo 1: $x = x + 2$;

1. $r1 = x$

2. $r1 = r1 + 2$

6. $x = r1$

hilo 2: $x = x * 3$;

3. $r2 = x$

4. $r2 = r2 * 3$

5. $x = r2$

memoria

$x=1$

$x=1$

$x=1$

$x=1$

$x=3$

$x=3$

resultado de los 20 intercalados

$x = 3: 18$

$x = 9: 1$

$x = 5: 1$

real, TD6 (8 hilos, 100 000 boletos):

tickets_race: 103.195 vendidos (tickets = -7)

tickets_mutex: 100.000 vendidos (tickets = 0)

tickets_sem: 100.000 vendidos (tickets = 0)

Operaciones atómicas

- Se ejecutan completas o no se ejecutan: **indivisibles**
- Típicamente: un acceso a memoria del tamaño nativo, una instrucción simple
- No atómicas: `x++` (leer-sumar-escribir), acceder a 64 bits en una máquina de 32

En C moderno

Desde C11, dos hilos que acceden a la misma variable sin sincronizar (y al menos uno escribe) es una **carrera de datos: comportamiento indefinido**. Para lo atómico: `_Atomic`, `<stdatomic.h>` o `__atomic_add_fetch` (lo usa `shm_fork`, unidad 6). `volatile` no hace nada atómico.

Demasiada leche: exclusión mutua por software

HILO 1	HILO 2
mira la heladera: no hay leche	
va a la tienda	mira la heladera: no hay leche
compra leche	va a la tienda
vuelve	compra leche... ¡demasiada!

- **Sincronización:** usar operaciones atómicas para coordinar hilos
- **Exclusión mutua:** un solo hilo hace la acción a la vez
- **Sección crítica:** el código que solo un hilo puede ejecutar a la vez
- Soluciones 1–3: cerrar la heladera, una nota, una nota por hilo... y sus fallas

Peterson y Dekker: correctos... en papel

C

```

/* Peterson, thread 1 */
postit_1 = true; turn = 2;
while (postit_2 && turn == 2)
    ;                               /* busy waiting */
if (!milk) milk++;                 /* critical section */
postit_1 = false;

```

- Correctos si los accesos a memoria son atómicos y **ordenados**
- Pero el compilador y el procesador **reordenan** (diapositiva 154)
- Y usan espera activa: consumen CPU

Salida real

```

$ for i in 1 2 3 4 5; do ./peterson; done; for i in 1 2 3 4 5; do ./peterson_fence; done
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
...
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
...

```

Todas las soluciones, todos los estados

hilo 0

```

postit_0 = 1
turn = 1
/* sin barrera */
loop: q = postit_1
if (!q) goto cs
t = turn
if (t ≠ 1) goto cs
goto loop
cs: enter()
r = milk
if (r) goto x
milk++
x: leave()
postit_0 = 0
end

```

hilo 1

```

postit_1 = 1
turn = 0
/* sin barrera */
loop: q = postit_0
if (!q) goto cs
t = turn
if (t ≠ 0) goto cs
goto loop
cs: enter()
r = milk
if (r) goto x
milk++
x: leave()
postit_1 = 0
end

```

memoria compartida

```

milk = 0
postit_0 = 0
postit_1 = 0
turn = 0

```

búfer del hilo 0: p0=1, turn=1

búfer del hilo 1: p1=1, turn=0

¡LOS DOS EN LA SECCIÓN CRÍTICA!

exploración exhaustiva

termina con 1 litro(s): 14 estado(s) final(es)

termina con 2 litro(s): 4 estado(s) final(es)

contraejemplo más corto: 12 pasos (paso 12)

Instrucciones atómicas: *test-and-set* y *compare-and-swap*

c

```
int test_and_set(int *p) {    /* atomic */
    int old = *p;  *p = 1;  return old;
}
/* spinlock: busy-waits until the old value is 0 */
void lock(int *l)   { while (test_and_set(l)) ; }
void unlock(int *l) { *l = 0; }
```

- x86: `xchg`, `lock cmpxchg`, `lock xadd`; ARM64: `ldxr` / `stxr` o `cas`
- La espera activa gasta CPU y puede causar **inversión de prioridad**
- ¿Un cerrojo sin espera? No. ¿Sin espera activa? En parte →

Espera activa limitada, hoy: el *futex*

Salida real

```
$ strace -f -c -e trace=futex ./lock_cost 1 100000; strace -f -c -e trace=futex ./lock_cost 8 100000
1 thread(s) x 100000 lock/unlock: counter = 100000, 48.4 ns per lock/unlock
  0.00    0.000000    0    1    futex
8 thread(s) x 100000 lock/unlock: counter = 800000, 85.5 ns per lock/unlock
100.00    0.223730    27    8013    3116 futex
```

- Mutex libre → una instrucción atómica en espacio de usuario, **sin llamada al sistema**
- Mutex ocupado → `futex(WAIT)`: el núcleo duerme al hilo y lo despierta en el `unlock`
- `pthread_spin_lock` nunca duerme (espera activa): solo para secciones muy cortas en las que dormir costaría más que girar

Cerrojos: pthread_mutex_lock / unlock

C tds/td6/tickets_mutex.c

```
static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

...
for (;;) {
    pthread_mutex_lock(&lock);
    if (tickets ≤ 0) {                /* checked w.
        pthread_mutex_unlock(&lock);
        break;
    }
    sold++;
    sched_yield();
    tickets--;
    pthread_mutex_unlock(&lock);
}
```

O dinámico: pthread_mutex_init(&m, attr)
 (pthread_mutexattr_t; NULL = por omisión) ...
 pthread_mutex_destroy(&m) .

Salida real

```
$ ./tickets_race 100000 | tail -3
...
103195 tickets sold out of 100000 (tickets = -7)
```

Salida real

```
$ ./tickets_mutex 100000 | tail -3
...
100000 tickets sold out of 100000 (tickets = 0)
```

Corrección (diapositiva 162): devuelve 0 o el número de error (EDEADLK solo en mutex con verificación), no -1.

Semáforos (Dijkstra, fines de los 60)

Un entero ≥ 0 con dos operaciones atómicas: **P** (*proberen*: espera a que sea positivo y resta 1) y **V** (*verhogen*: suma 1 y despierta).

- Valor inicial 1 → cerrojo
- Valor inicial 0 → esperar a otro hilo
- Valor inicial k → a lo sumo k a la vez

c

```
int sem_init(sem_t *s, int pshared, unsigned value);
int sem_destroy(sem_t *s); /* unnamed */
sem_t *sem_open(const char *name, int flags /* O_CREAT [| O_EXCL] */,
                mode_t mode, unsigned value); /* SEM_FAILED */
int sem_close(sem_t *s); /* named: this process */
int sem_unlink(const char *name); /* named: the name */
int sem_wait(sem_t *s); /* P */
int sem_post(sem_t *s); /* V */
```

Corrección (diapositiva 167): `sem_open` devuelve `SEM_FAILED` (no -1) y no tiene valor por omisión: con `O_CREAT` hay que dar `value`. Y `sem_close` no borra el semáforo: el nombre sigue en `/dev/shm` hasta `sem_unlink`.

El problema del productor y los consumidores

- Un productor escribe datos en un búfer compartido de tamaño limitado
- Los consumidores los sacan
- El productor espera si el búfer está lleno; el consumidor, si está vacío
- El búfer se modifica: hace falta sección crítica

Ejemplos: una tubería de comandos (unidad 5), `gcc (cpp | cc1 | as | ld)`, filtros de imágenes, colas de mensajes entre servicios.

Salida real

```
$ ./prodcons_multi 4 100000  
consumer 2: 23918 items  
consumer 3: 26423 items  
consumer 1: 23037 items  
consumer 0: 26622 items  
sum = 5000050000, expected 5000050000: OK
```

Dos semáforos y un mutex, paso a paso

productor

```
in = (in+1) % BUF
v = 32 · ejecuta
```

búfer (BUF = 8)



empty = 5 full = 0 out_lock: C2

consumidor 1 · 8

```
wait(full)
espera: full = 0
```

consumidor 2 · 5

```
unlock(out_lock)
ejecuta
```

consumidor 3 · 9

```
wait(full)
espera: full = 0
```

consumidor 4 · 7

```
lock(out_lock)
espera el mutex
```

los 40 valores: verde = una vez · rojo = perdido · naranja = repetido



real, prodcons_multi 4 100000: valores por consumidor



Variables de condición

C

```
pthread_mutex_lock(&mutex);  
while (!condition)           /* while, not if */  
    pthread_cond_wait(&cond, &mutex);  
/* condition is true, mutex held */  
pthread_mutex_unlock(&mutex);  
/* whoever changes the condition: */  
pthread_cond_signal(&cond);   /* wakes one */  
pthread_cond_broadcast(&cond); /* wakes all */
```

- Esperar una condición cualquiera (no solo un contador)
- `cond_wait` suelta el mutex y duerme **de forma atómica**, y lo vuelve a tomar al despertar
- `while` y no `if`: puede haber despertares espurios u otro hilo pudo cambiar la condición

Interbloqueos

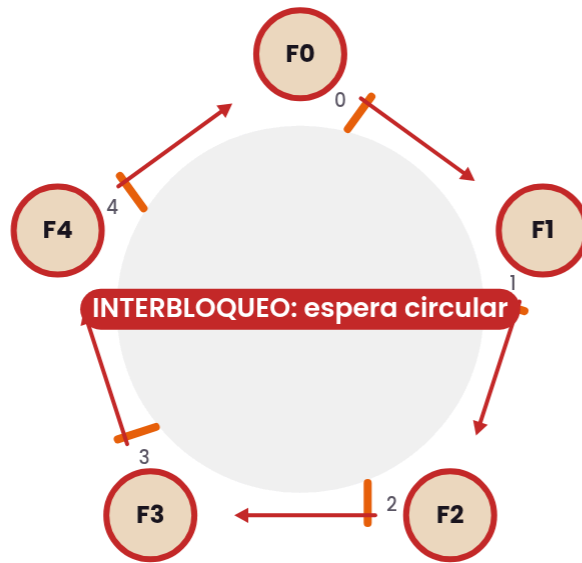
C

```
/* thread 1 */      /* thread 2 */
x.P();              y.P();
y.P();              x.P();  /* ← may never return */
/* ... */           /* ... */
y.V(); x.V();        x.V(); y.V();
```

No siempre ocurre: depende de la ejecución.
Hace falta más de un recurso y no se resuelve
sin intervención externa.

- Recursos: CPU, memoria, disco, dispositivos; compartibles o no; expropiables o no
- Ejemplo: un cruce con prioridad a la derecha, cada auto con un tramo y esperando el siguiente
- Salidas: retroceder (*rollback*), quitar un recurso (expropiar)... con riesgo de inanición

Los filósofos comensales



- piensa
- tiene hambre
- come
- terminó
- espera a (grafo)

condiciones de Coffman

- ✓ exclusión mutua
- ✓ retener y esperar
- ✓ sin expropiación
- ✓ espera circular

Las cuatro condiciones y cómo prevenirlas

1. Exclusión mutua
 2. Retener recursos mientras se espera otros
 3. Sin expropiación
 4. Espera circular: T_1 espera a T_2 , ..., T_n espera a T_1
- **Orden global** de adquisición (rompe 4): la solución del TD6
 - Expropiar (rompe 3), con riesgo de estado inconsistente
 - Detectar ciclos en el grafo de asignación; evitarlos con el algoritmo del banquero
 - Tiempos límite (`trylock`, `timedlock`): sin garantía

Conclusión

- Problemas: resultados no deterministas, interbloqueos, inanición
- Toda escritura en un recurso compartido → en una sección crítica
- La exclusión mutua por software funciona en papel; en el hardware real, **mutex, semáforos y atómicos**
- Los interbloqueos son difíciles de reproducir y exigen las cuatro condiciones

TD6 resuelto + desafío con IA

- `threads`, `tickets_race/mutex/sem`
- `prodcons`, `prodcons_multi`, `philosophers` (y su versión que se bloquea)
- Nuevo: `create_cost`, `peterston`, `lock_cost`

Bash

```
make -C tds && tds/tests/run.sh td6
```



Universidad
Indoamérica

PARA LLEVAR

Tres preguntas

1. ¿Por qué `tickets_race` vende más boletos de los que hay?
2. ¿Por qué Peterson falla en un x86 y qué lo arregla?
3. ¿Qué condición rompe el orden global de los palillos?